Chapter 4: Implementation

This chapter explains the steps taken to turn the Chapter 3 methodology into a working software tool and making it suitable for real-time use. The software development decisions are directly tied to the main research goals of the project: sub-10 ms latency, encoder responsiveness, and reproducible training.

4.1 Environment and compatibility

The BRAVE resynthesis pipeline (Caillon & Esling, 2021) was implemented on a single computer. The complete software and hardware setup is shown in Table 4.1.

Table 4.1. Hardware and software environment used for all training and deployment in this project.

Component	Version
Operating system	Windows 11 (build 26200+)
Python	3.14.3
PyTorch	2.11.0+cu128
CUDA driver	13.2
GPU	NVIDIA RTX 5080 (16 GB VRAM)
acids-rave	2.3.1
pytorch-lightning	2.6.1
scipy	≥ 1.14
Audio tooling	librosa, soundfile, ffmpeg
Real-time host	Pure Data 0.55 with the nn~ external

When training, the RTX 5080 demonstrated a throughput capacity of 9-12 optimization steps per second on average. As a result, each full 3,000,000-step experiment required 90–120 hours of wall-clock time, making GPU availability the dominant limiter for the pacing of iteration.

A notable implementation challenge came from library drift in acids-rave 2.3.1, which was originally authored against earlier SciPy and PyTorch Lightning ecosystems. As a result, four library source patches were made to accommodate these major changes, grouped into three categories: two SciPy API updates (a relocated scipy.signal.kaiser import and a type-coerced kaiserord argument), one SciPy parameter rename (firwin's nyq argument to fs), and one PyTorch Lightning 2.x API rename (validation_epoch_end to on_validation_epoch_end).

Altogether the changes amount to less than ten lines of code. The exact diffs are made available in Appendix C, as well as in the public repository for independent verification of the environment.

4.2 Training Pipeline

The use of the .gin configuration system to direct training made it easy to modularly and hierarchically override hyperparameters. Rather than viewing each configuration as an individual artefact, the work developed a configuration chain where each step is a reaction to the failure mode of the previous iteration. A summary of this is given in Table 4.2.

Table 4.2. The progression of configuration changes across guitar iterations and the second drums iteration. Each row specifies the major change made and the failure mode it was meant to cure.

Iteration	Configuration	Main change introduced	Motivation
guitar_v1	unmodified c128_r10.gin	— (baseline)	first run; revealed posterior collapse
guitar_v2	c128_r10_beta_fixed.gin	Log-space β warmup (1×10^{-4} → 0.1)	Eliminate posterior collapse
guitar_v3, guitar_v4	c16_r10_beta_fixed.gin	LATENT_SIZE reduced 128 → 16	Address 0.005 nats-per-dimension utilisation
guitar_v5	c16_r10_v5_balanced.gin	Discriminator capacity halved and shallower	Counter discriminator dominance observed in v4
guitar_v6	guitar_v6_overrides.gin (on official v2.gin + causal.gin)	Adopt official architectural baseline; extend Phase 1 to 1.5 M steps	Replace manual architectural tuning with a community-validated configuration
drums_v2	drums_v2_overrides.gin	Apply the v6 template to the Groove corpus	Extend the architectural baseline to the drums case study (training initiated; terminated before completion — discussed in §5.X)

The way to handle disruptions during multi-day runs is that every iteration is initiated with a Windows .bat script. The script calls PowerShell to locate the latest checkpoint that matches the run name; if a checkpoint is found, the --ckpt flag is used to resume training, otherwise, a new training is initiated. This system made the pipeline safely interruptible and was used several times to recover after both planned and unplanned system restarts.

The checkpointing method had --val_every 10000, which means that the model state was saved every ten thousand optimization steps. Three levels of redundancy were kept: the RAVE-native best.ckpt (selected by lowest validation reconstruction loss), the epoch-tagged files (e.g. epoch=2511.ckpt), and fixed snapshots every 500, 000 steps. As mentioned in §3.2, such redundancy came in handy with the guitar_v6, whose reconstruction quality reached a maximum around epoch 1935 of Phase 2 and slightly went down toward the last epoch. Hence, the the canonical model guitar_v6_best.ts is the mid-trajectory checkpoint of Phase 2 rather than the last one.

Reproducibility. Version control for all config files, preprocessing scripts, and evaluation tools is maintained in the project repository, and everything will be released as open source alongside the thesis. The repository is structured so that the documented iterations can be independently regenerated through the same configurations and preprocessing pipeline.

4.3 Real-Time Deployment

Generally, model deployment was based on the standard acids-rave export flow:

rave export --run <RUN_DIR> --name <NAME>

This command converts the trained PyTorch graph into a TorchScript.ts file containing the learned weights along with the cached convolutional state necessary for incremental inference execution.

The inference environment is based on the nn~ Pure Data external which was selected primarily for two reasons that are directly linked to this project's research goals. Firstly, nn~ is a well-established, continually-supported deployment target which enables the sub-10 ms latency budget as per §3.1 to be met readily without sacrificing valuable development time on a custom C++ audio engine. Secondly, since Pure Data comes with DSP blocks (gain, limiters, filtering) readily available, it is very easy to correct for any quirks of the trained model through these, without having to add extra inference stages to the audio path.

The deployed signal chain follows a linear path:

adc~ 1 → nn~ <model>.ts forward → *~ <gain> → dac~

We provide a reference block diagram in Figure 3.1. The audio from the microphone is connected as an input to the nn~ .ts forward method, which processes a tensor of shape (1, 1, 64) representing each audio block.

Sample-rate compatibility. One of the challenges to reproduce the result revealed through deployment testing was related to the Pure Data audio engine's default sample-rate. Using the default 48,000 Hz resulted in silent or unusable output since the 44,100 Hz used during training and LMDB construction was not matched. Thus, the necessity to explicitly set the sample rate to 44,100 Hz in Media → Audio settings is recognized as a crucial deployment step and is documented accordingly.

Gain compensation. Output amplitudes from BRAVE decoder typically range between 0.005 and 0.05. Therefore, the unscaled output of nn~ is generally below practical listening level when connected directly to dac~. The fixed multipliers were determined by trial and error with reference to the actual output amplitude of each model: ×50 for lower-energy outputs such as drums_v1, and ×20 to ×30 for the higher-energy guitar_v6. The compensation has been done through a single *~ multiplier in the audio path to raise the output to an audible listening level without adding extra DSP stages that could reduce the latency budget.

4.4 Evaluation Tooling

Some diagnostic tools were designed to measure model behavior quantitatively apart from informal listening. A tailored Python program, scripts/analyze_noise_floor.py, calculates the 5th percentile of 100 ms RMS segments for each piece of audio in the guitar database and then collates the results per source. By executing this program on all 899 files, the per-source noise-floor distribution presented in §3.3 was obtained, showing among other things a 35.8 dB difference between electric DI and microphone-captured acoustic sources.

The responsiveness of each candidate checkpoint was measured by a separate synthetic-input test. The program sends five spectrally diverse signals, silence, three sine tones (220, 440, and 880 Hz), a frequency sweep, and white Gaussian noise, through the model and calculates the mean absolute differences of latent vectors and audio outputs. These differences are then evaluated against the health thresholds set in §3.4. The thresholds were determined by comparing responsive and collapsed checkpoints during early exploratory runs. Collapsed models yielded pairwise latent differences under 0.2 and pairwise output differences under 0.005, while functionally responsive checkpoints always showed values above 0.5 and 0.01 respectively. The very same program marked the early collapsed versions, guitar_v1 (output difference = 0.006) and guitar_v2 (output difference = 0.003), as well as the partial-response state of guitar_v3 (output difference = 0.042).

Detailed training health records were kept via TensorBoard (tensorboard --logdir runs/). Important scalars such as KL divergence and the discriminator logit gap between pred_real and pred_fake were continuously monitored. This measurement setup also made it possible to detect early in training the situation when the

discriminator dominated guitar_v4 (gap = 5.65) and the one when guitar_v5 failed in the opposite way, both logits becoming negative and the gap decreasing to about 0.8.

Combined, these tools turned the process of implementing the model from one that was basically trial and error with trying out different architectures into a reproducible and diagnosable training pipeline that meets the latency and responsiveness requirements for live musical interaction (Wessel & Wright, 2002). The diagnostics and checkpoint analyses that came out of this process became the foundation of the comparative evaluation in Chapter 5.

References used in this chapter

- Caillon, A., & Esling, P. (2021). RAVE: A variational autoencoder for fast and high-quality neural audio synthesis. *arXiv preprint arXiv:2111.05011*.
- Wessel, D., & Wright, M. (2002). Problems and prospects for intimate musical control of computers. *Computer Music Journal*, *26*(3), 11–22.

(Full bibliography maintained globally at the end of the thesis.